



Cesto

Manual técnico do projeto

API de e-commerce em microsserviços com TypeScript: instalação, telas, arquitetura, OAuth2, contratos tipados, API, configuração, extensão, testes e solução de problemas.

Copyright (©) 2026 Clara A. Portes

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

1. Visão geral **3**
Para que serve 3
O que o Cesto faz 3
Tecnologias 3
Identidade visual 4

2. Como rodar **5**
Requisitos 5
Primeira execução 5
Contas de demonstração 5
Comandos do dia a dia 5
Uso de verdade 5
Acesso pela rede e HTTPS 6

3. Guia das telas **7**
Loja 7
Entrada e cadastro 8
Carrinho e checkout 8
Pedidos 9
Notificações 10
Administração 10
No celular 11

4. Arquitetura **12**
Serviços 12
Caminho de uma requisição 12
A saga do pedido 12
Outbox, deduplicação e novas tentativas 13
Estrutura de pastas 13

5. Autenticação OAuth2 **14**
Fluxos suportados 14
Passo a passo do login 14
Escopos e papéis 14
Chaves e tokens 15

6. Contratos e tipagem **16**
DTOs genéricos 16
Contratos de rota 16
Erros de domínio 16
Eventos tipados 17

7. API REST **18**
Envelope e convenções 18
Rotas 18
Exemplos 19

8. Configuração **20**
Variáveis do .env 20
Checklist para produção 20

9. Personalizar e estender **22**
Adicionar uma rota 22
Adicionar um evento 22
Adicionar um serviço 22
Trocar o pagamento simulado 22
Imagens, marca e interface 22

10. Testes e qualidade **23**
Testes automatizados 23
Integração contínua 23

11. Solução de problemas **24**
Problemas comuns 24
Onde olhar 24

CAPÍTULO 1

Visão geral

Para que serve

O Cesto é uma API de e-commerce dividida em microsserviços e escrita em TypeScript. Ela cobre o caminho completo de uma loja: catálogo, estoque, carrinho, pedido, pagamento e notificações. Uma interface web em React acompanha o projeto e usa a mesma API que qualquer outro cliente usaria.

Ele foi feito para três usos: servir de base para uma loja real, servir de referência de arquitetura (eventos, saga, outbox, OAuth2, contratos tipados) e servir de material de estudo. Tudo sobe com um comando e vem com dados de demonstração.

O que o Cesto faz

- **Oito serviços independentes**, cada um com processo e banco próprios.
- **Pedido como saga**: reserva de estoque, cobrança e confirmação, com desfazimento automático em caso de falha.
- **Eventos confiáveis** em Redis Streams, com outbox transacional, deduplicação, novas tentativas e dead-letter.
- **OAuth2**: Authorization Code com PKCE, Client Credentials, JWT RS256, JWKS, refresh token com rotação e detecção de reuso.
- **Contratos tipados** com Zod: o mesmo contrato valida, tipa o handler, gera o OpenAPI e alimenta o cliente HTTP.
- **Erros de domínio** mapeados para HTTP num só lugar.
- **Loja web** com catálogo, carrinho, checkout, pedidos, notificações e painel administrativo.

Tecnologias

Camada	Tecnologia	Papel
Linguagem	TypeScript 5.9 (modo estrito)	Todo o back-end, os contratos e a interface
Execução	Node.js 22	Processo de cada serviço
HTTP	Fastify 5, @fastify/http-proxy, @fastify/rate-limit	Serviços, gateway e limite de requisições
Validação	Zod 4	Entrada, saída, eventos e configuração
Persistência	Prisma 7 + PostgreSQL 16 (adapter pg)	Um banco por serviço, migrações versionadas
Mensageria	Redis 7 Streams + ioredis	Eventos, carrinho e rate limit
Segurança	jose (JWT RS256, JWKS), scrypt	Tokens e senhas
Interface	React 19, Vite 7, Lucide, Public Sans	Loja e painel
Infraestrutura	Docker Compose, Nginx	Execução local e em servidor
Qualidade	Vitest, teste ponta a ponta, GitHub Actions	Testes e integração contínua

Identidade visual

A marca é um cesto montado com blocos, que representa os serviços independentes formando uma loja. A logo é preta e branca, com cantos retos. A interface e este manual usam a paleta do projeto:

Cor	Hex	Uso
Papel	`#f4f1de`	Fundo
Tinta	`#1b2127` / `#2b3540`	Texto, barra superior e botões
Rosa	`#d8a7b1` / `#a65a6a`	Destaques, alertas e cancelamentos
Sálvia	`#b8c9a9` / `#5e7350`	Estados positivos
Ardósia	`#8395a7`	Elementos secundários

CAPÍTULO 2

Como rodar

Requisitos

Para usar com Docker, basta o **Docker** com Docker Compose (no Windows, o Docker Desktop). Para desenvolver sem Docker: Node.js 22, PostgreSQL 16 e Redis 7.

O computador precisa de cerca de 2 GB de memória livre para os onze contêineres.

Primeira execução

1. Baixe ou clone o repositório.
2. No Windows, dê dois cliques em `iniciar.cmd`. No Linux ou macOS, rode `./iniciar.sh`.
3. O script cria o `.env` a partir do `.env.example`, com uma senha aleatória para o banco e um segredo aleatório para os serviços.
4. As imagens são construídas e os contêineres sobem: PostgreSQL, Redis, os oito serviços e a interface.
5. Cada serviço aplica as próprias migrações ao iniciar. O `auth` cria os clientes OAuth2 e as contas de demonstração, e o `catalog` publica os produtos de exemplo.
6. O navegador abre em **http://localhost:8080**.

> A primeira construção baixa as dependências e leva alguns minutos. As seguintes reaproveitam o cache.

Contas de demonstração

Papel	E-mail	Senha	Pode
Administrador	`admin@cesto.dev`	`Cesto@2026`	Tudo, incluindo produtos, estoque e expedição
Cliente	`cliente@cesto.dev`	`Cesto@2026`	Carrinho, pedidos e notificações

Os nomes, os produtos e o domínio `cesto.dev` são fictícios. Para trocar a senha das contas de demonstração, altere `DEMO_PASSWORD` antes da primeira execução.

Comandos do dia a dia

```
docker compose ps                          # estado dos contêineres
docker compose logs -f orders inventory    # acompanhar a saga
docker compose restart payments            # reiniciar um serviço
docker compose up -d --scale inventory=3   # mais consumidores de eventos
docker compose down                        # parar (mantém os dados)
docker compose down -v                     # parar e apagar bancos e Redis
```

Uso de verdade

1. No `.env`, defina `SEED_DEMO=false` antes da primeira execução, com o banco vazio.

2. Ajuste `WEB_REDIRECT_URIS` e `CORS_ORIGINS` com o endereço público, por exemplo `https://loja.exemplo.com/callback`.
3. Suba o projeto e cadastre sua conta pela tela de entrada.
4. Promova a conta a administrador no banco `cesto_auth`:

```
docker compose exec postgres psql -U cesto -d cesto_auth \
  -c "UPDATE users SET role = 'ADMIN' WHERE email = 'voce@empresa.com';"
```

Saia e entre de novo para receber um token com os escopos de administrador.

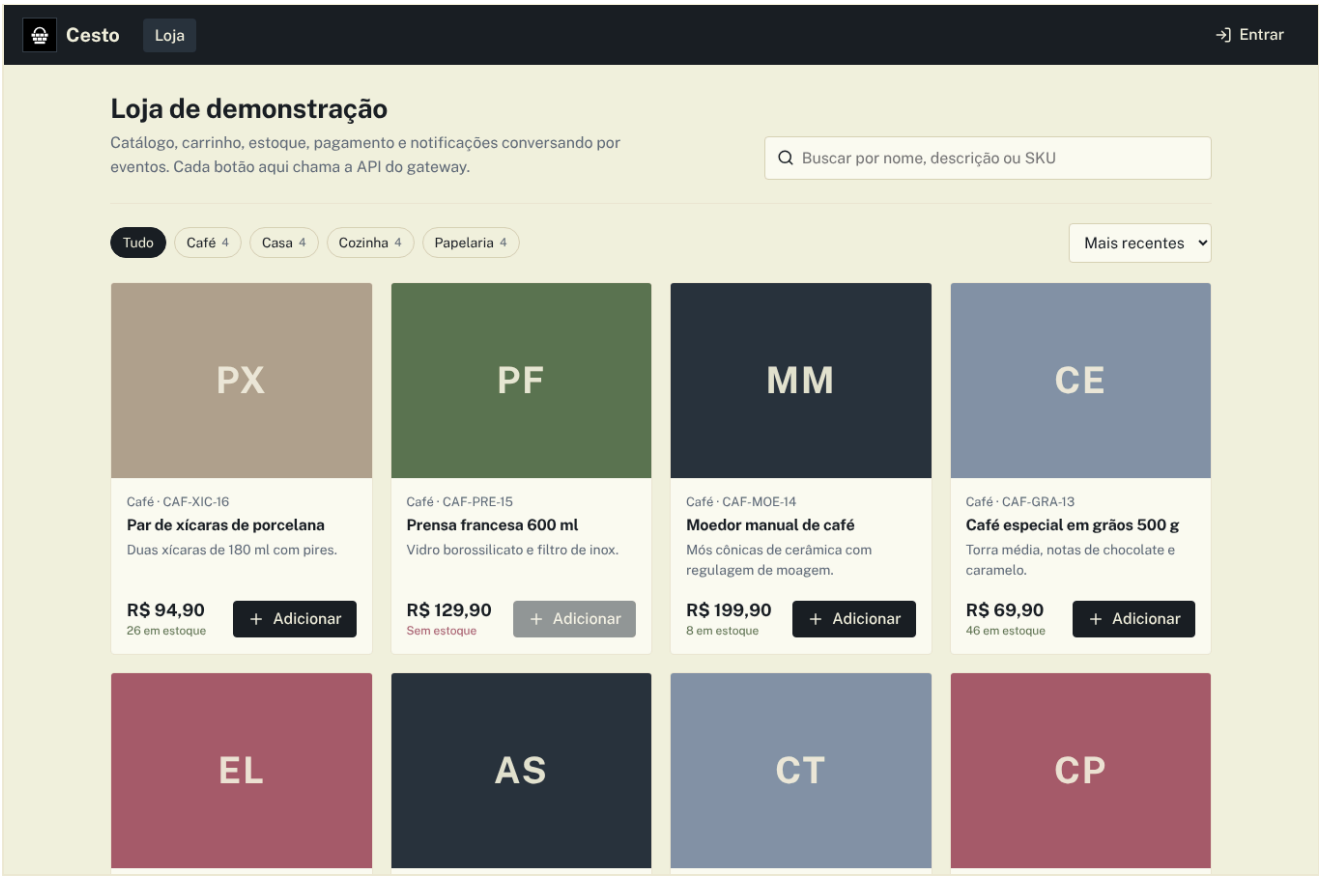
Acesso pela rede e HTTPS

Coloque um proxy com HTTPS (Caddy, Traefik ou Nginx) na frente da porta `APP_PORT`. O contêiner `web` já encaminha `/api`, `/oauth`, `/.well-known` e `/health` para o gateway. O gateway confia nos cabeçalhos `X-Forwarded-*` para registrar o IP real e aplicar o rate limit por cliente.

CAPÍTULO 3

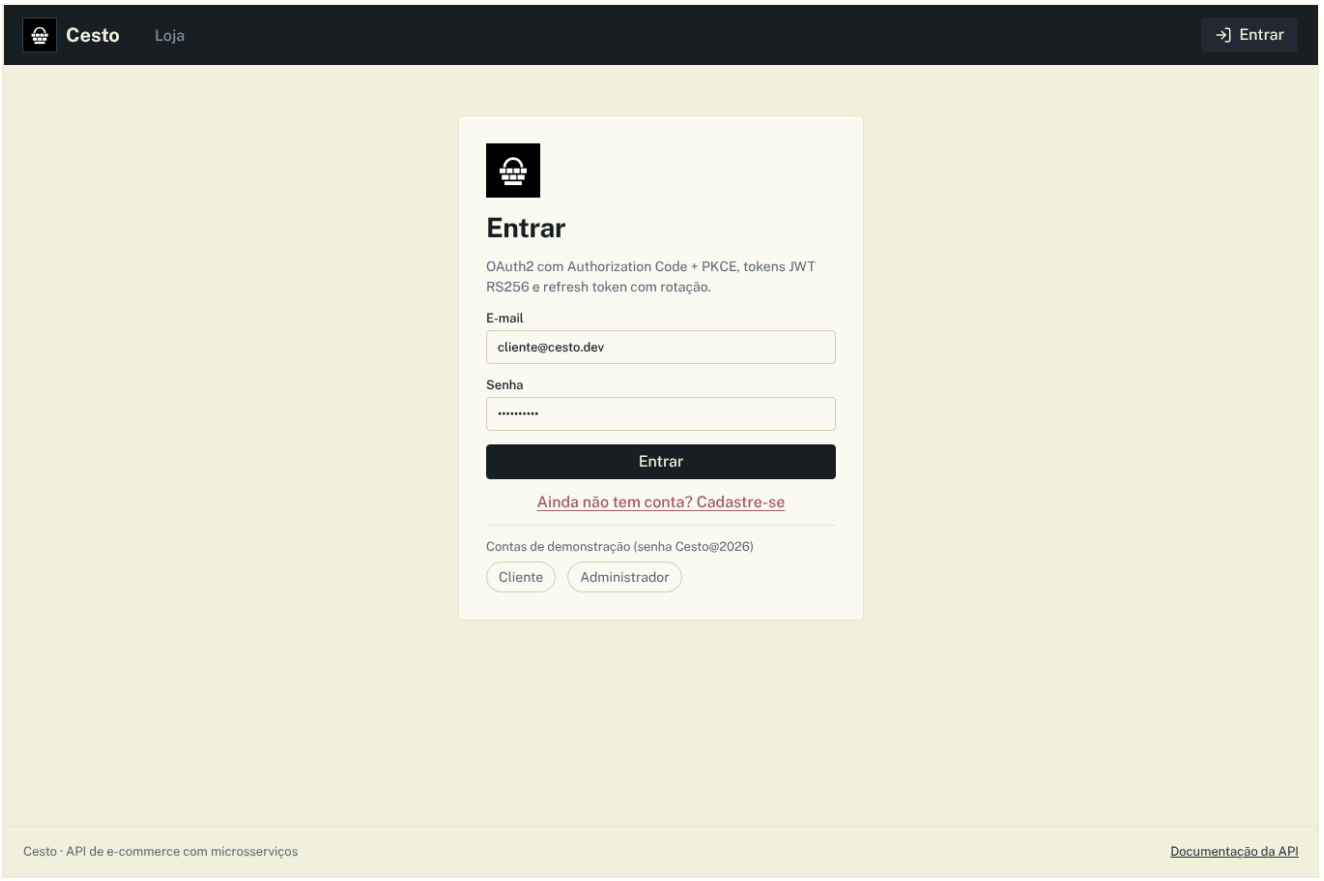
Guia das telas

Loja



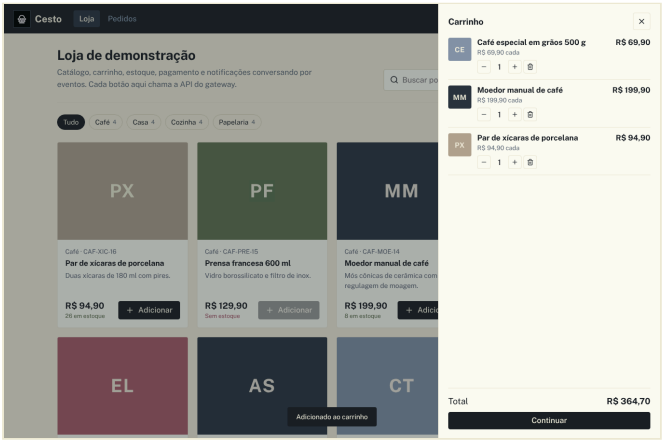
Lista os produtos ativos com busca por nome, descrição ou SKU, filtro por categoria, ordenação e paginação. O saldo de cada produto vem do serviço de estoque. Produtos sem estoque continuam visíveis, mas não podem ser adicionados.

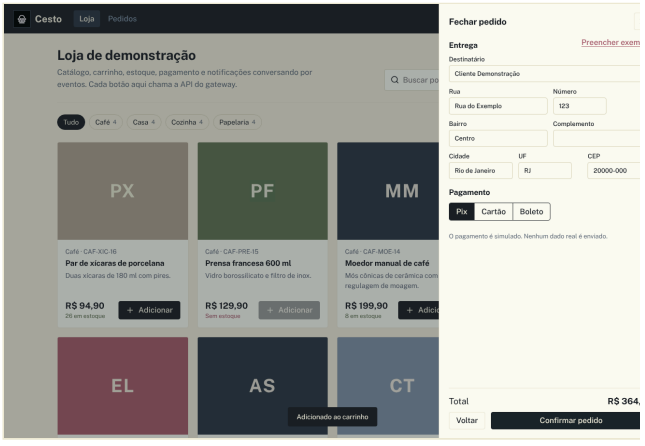
Entrada e cadastro



A tela de entrada faz o fluxo Authorization Code com PKCE: gera o `code_verifier`, envia o desafio S256 com o e-mail e a senha, recebe o código e troca pelo token. O cadastro cria um cliente e já faz o login.

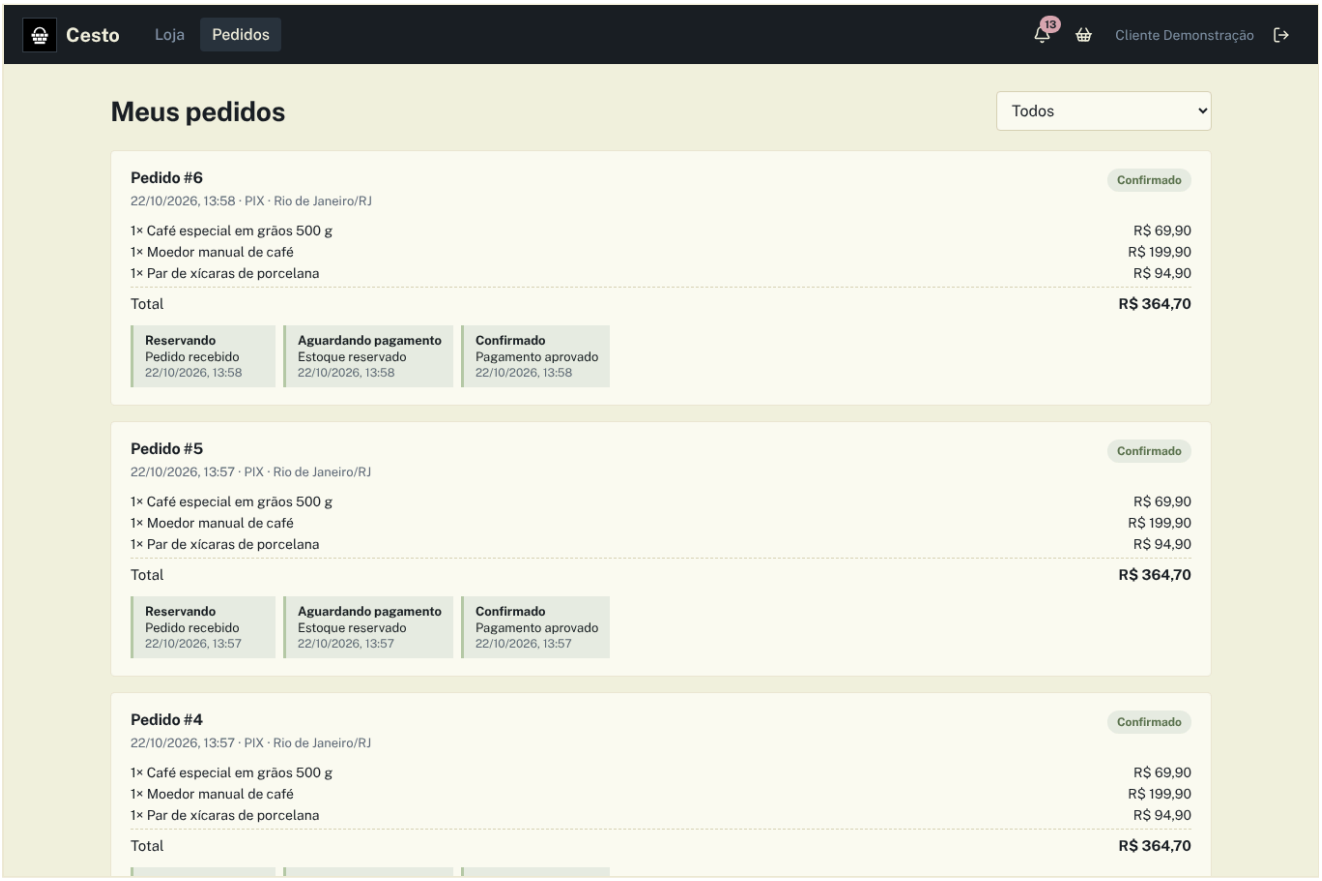
Carrinho e checkout





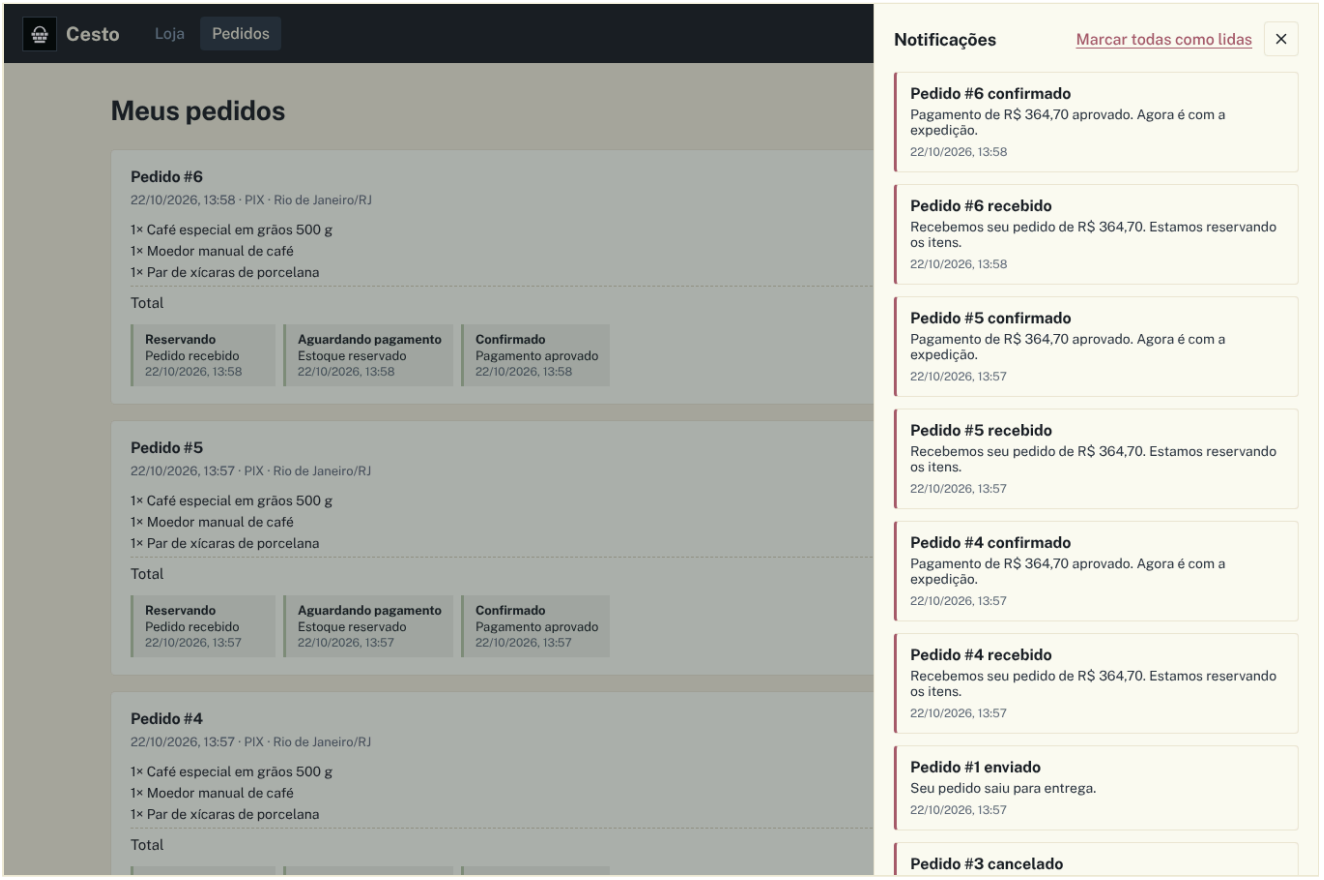
O carrinho guarda só o produto e a quantidade. Preços e nomes são lidos do catálogo a cada consulta, então o cliente sempre vê o valor atual. No checkout, escolha Pix, Cartão ou Boleto. Com Cartão é possível simular um cartão recusado.

Pedidos



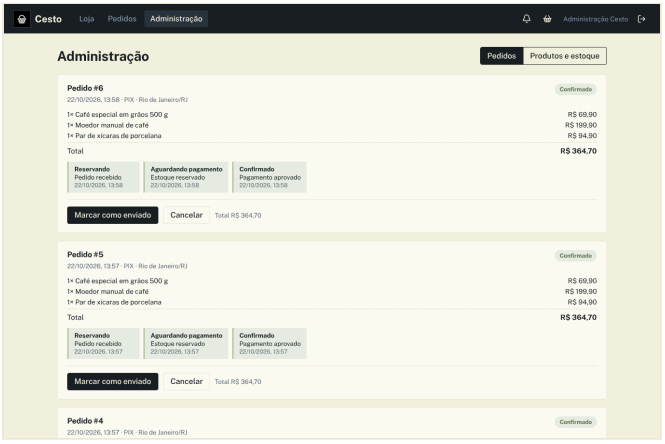
Cada pedido mostra os itens, o total e a linha do tempo com cada etapa da saga. A tela se atualiza sozinha enquanto há pedidos em andamento. Pedidos ainda não confirmados podem ser cancelados pelo cliente.

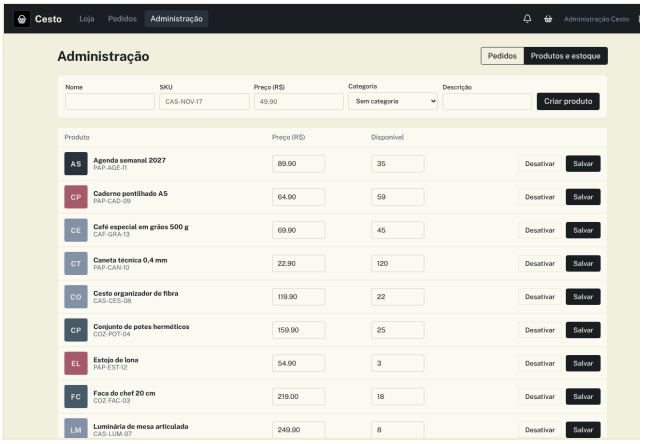
Notificações



Cada evento importante vira uma notificação: boas-vindas, pedido recebido, confirmado, cancelado, pagamento recusado, enviado e entregue. O sino mostra quantas ainda não foram lidas.

Administração





Visível só para administradores. Em **Pedidos**, a loja vê os pedidos de todos os clientes e marca como enviado, entregue ou cancelado. Em **Produtos e estoque**, cria produtos, altera preço, ativa ou desativa e ajusta o saldo disponível.

No celular



A interface se adapta a telas pequenas.

CAPÍTULO 4

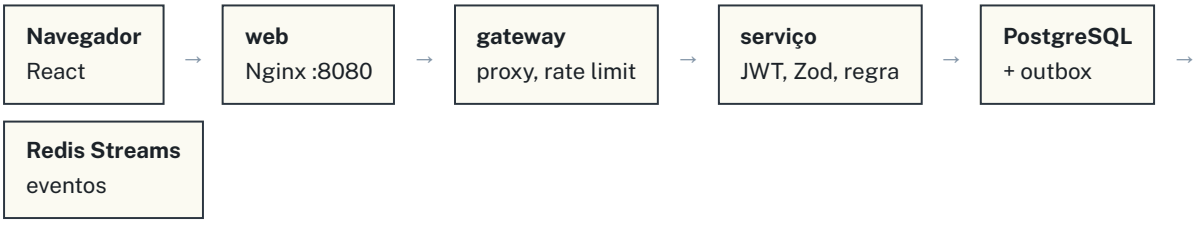
Arquitetura

Serviços

Serviço	Porta	Banco	Publica	Consome
gateway	8000	—	—	—
auth	4001	cesto_auth	user.registered	—
catalog	4002	cesto_catalog	product.created, product.updated	—
inventory	4003	cesto_inventory	inventory.reserved, inventory.rejected	product.*, order.placed, order.confirmed, order.cancelled
cart	4004	Redis	—	order.placed
orders	4005	cesto_orders	order.*, payment.requested	inventory.*, payment.approved, payment.declined
payments	4006	cesto_payments	payment.approved, payment.declined	payment.requested, order.cancelled
notifications	4007	cesto_notifications	—	user.registered, order.*

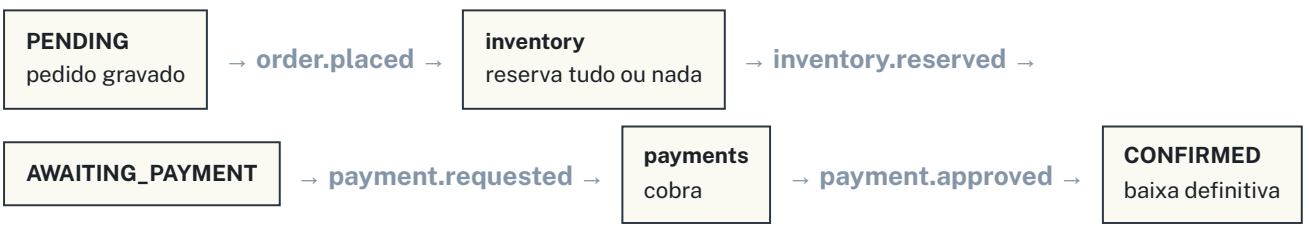
Cada serviço é dono dos seus dados. Nenhum serviço lê o banco de outro: quando precisa de um dado alheio, chama a API interna com um token de serviço ou reage a um evento.

Caminho de uma requisição



O gateway acrescenta um `x-request-id` que acompanha a requisição em todos os logs e volta na resposta. Rotas `/internal/*` nunca são expostas pelo gateway.

A saga do pedido



Situação	O que acontece
Falta estoque	`inventory.rejected` cancela o pedido sem mexer no saldo
Pagamento recusado	`payment.declined` cancela o pedido e `order.cancelled` devolve a reserva
Cliente cancela antes do pagamento	O pedido é cancelado e a reserva volta para o saldo
Loja cancela um pedido confirmado	O estoque baixado volta e o pagamento é estornado
Pagamento aprovado depois do cancelamento	O pedido continua cancelado e o pagamento é estornado

Outbox, deduplicação e novas tentativas

- **Outbox:** o evento é gravado na tabela `outbox_events` na mesma transação da mudança. Um relay publica no Redis e marca como publicado. Se o Redis cair, nada se perde.
- **Deduplicação:** quem consome grava o `id` do evento em `processed_events` na mesma transação do efeito. Entregas repetidas são ignoradas.
- **Grupos de consumidores:** cada serviço lê com `XREADGROUP`. Várias réplicas dividem o trabalho.
- **Novas tentativas:** mensagem não confirmada é reivindicada com `XCLAIM` depois de 15 s. Após 5 entregas, vai para `cesto:events:dead-letter` com o motivo.
- **Concorrência no estoque:** a reserva usa `UPDATE ... WHERE available >= quantidade` numa transação, e o banco ainda impede saldo negativo com `CHECK`.

Estrutura de pastas

```
packages/contracts/   esquemas Zod, DTOs genéricos, rotas, eventos, cliente tipado, dados de
demonstração
packages/core/        createService, route, verificação de JWT, erros de domínio, outbox, OpenAPI,
paginação
packages/bus/         EventBus em Redis Streams
services/<nome>/src/  main.ts (composição), env.ts (configuração), regras do domínio
services/<nome>/prisma/  schema.prisma e migrações
services/<nome>/test/ testes do serviço
apps/web/src/         App, páginas, componentes, cliente da API
docker/               Dockerfiles, nginx.conf, start.sh, init.sql
scripts/              dev.mjs, migrate.mjs, generate.mjs, smoke.mjs
```

CAPÍTULO 5

Autenticação OAuth2

Fluxos suportados

Fluxo	Quem usa	Detalhes
Authorization Code + PKCE (S256)	Interface web (`cesto-web`, cliente público)	Código de uso único, válido por 60 s, preso ao `redirect_uri` e ao desafio
Refresh Token	Interface web	Rotação a cada uso; reuso de um token antigo revoga a família inteira
Client Credentials	Serviços (`cesto-internal`)	Escopo `internal`, token em cache até 30 s antes de expirar

Passo a passo do login

1. O cliente gera um `code_verifier` aleatório e calcula `code_challenge = BASE64URL(SHA256(verifier))`.
2. `POST /oauth/authorize` com `client_id`, `redirect_uri`, `code_challenge`, `state`, e-mail e senha. A resposta traz o `code` e o mesmo `state`.
3. `POST /oauth/token` com `grant_type=authorization_code`, o `code` e o `code_verifier`.
4. A resposta traz `access_token` (JWT, 15 min), `refresh_token` (30 dias) e os escopos concedidos.

```
{ "access_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6...", "token_type": "Bearer",
  "expires_in": 900, "refresh_token": "...", "scope": "orders:read orders:write cart:write
notifications:read" }
```

O endpoint de token aceita JSON ou `application/x-www-form-urlencoded` e responde erros no formato do RFC 6749 (`{"error": "invalid_grant", "error_description": "..."}`).

Escopos e papéis

Escopo	Cliente	Administrador	Libera
`cart:write`	sim	sim	Alterar o carrinho
`orders:read`	sim	sim	Ver pedidos e pagamentos
`orders:write`	sim	sim	Fechar e cancelar pedidos próprios
`notifications:read`	sim	sim	Notificações
`catalog:write`	—	sim	Produtos e categorias
`inventory:write`	—	sim	Ajustar estoque
`orders:manage`	—	sim	Enviar, entregar e cancelar qualquer pedido
`internal`	—	—	Rotas internas (só serviços)

Cada rota declara no contrato o tipo de acesso (`public`, `user`, `admin` ou `service`) e os escopos exigidos. A verificação acontece antes do handler.

Chaves e tokens

- As chaves RSA de 2048 bits são geradas na primeira execução e guardadas em `signing_keys`. A chave pública fica em `GET /oauth/jwks`.
- Os demais serviços validam os tokens com o JWKS (cache de 10 min), sem consultar o `auth` a cada requisição.
- Senhas e segredos de cliente usam scrypt com sal aleatório. Códigos e refresh tokens são guardados só como SHA-256.
- Códigos e tokens expirados são apagados a cada hora.

CAPÍTULO 6

Contratos e tipagem

DTOs genéricos

Tipo	Para que serve
`Dto<S>` / `DtoInput<S>`	Tipo de saída e de entrada inferidos de um esquema Zod
`CreateDto<T, K>` / `UpdateDto<T, K>`	Tira `id`, datas e campos do servidor; o de atualização deixa tudo opcional
`Paginated<T>` + `paginated(schema)`	Página com itens, total e número de páginas, como tipo e como esquema
`ApiResponse<T>` + `envelope(schema)`	União discriminada `{ ok: true, data }` ou `{ ok: false, error }`
`Result<T, E>`	Retorno de sucesso ou falha sem exceção
`Brand<T, B>`, `DeepPartial<T>`, `Merge<A, B>`	Utilitários para modelar o domínio

Contratos de rota

Cada rota é declarada uma vez com `defineEndpoint`. O caminho vira tipo: `PathParams<'/orders/:id'>` resulta em `{ id: string }`. A partir do contrato saem `EndpointRequest<E>`, `EndpointBody<E>` e `EndpointResponse<E>`.

```
export const api = {
  catalog: {
    getProduct: defineEndpoint({
      method: 'GET', path: '/api/v1/catalog/products/:id', summary: 'Detalhe do produto',
      tags: ['catalog'], auth: 'public', params: IdParams, response: ProductDto
    })
  }
} as const;

route(app, api.catalog.getProduct, ({ params, principal }) => catalog.get(params.id,
isAdmin(principal)));

const product = await request(api.catalog.getProduct, { params: { id } });
```

`route` valida params, query e body com Zod, converte tipos (`?page=2` vira número), checa o token, o papel e os escopos, chama o handler, valida a saída e devolve no envelope. Campos extras são recusados (`z.strictObject`).

Erros de domínio

Código	HTTP	Quando
`VALIDATION_FAILED`	422	Entrada inválida, com a lista de campos
`UNAUTHENTICATED`	401	Sem token, token inválido ou expirado

Código	HTTP	Quando
`FORBIDDEN`	403	Papel ou escopo insuficiente (`details.missing`)
`NOT_FOUND`	404	Registro inexistente ou de outro usuário
`CONFLICT`	409	Duplicidade (SKU, slug, e-mail)
`INSUFFICIENT_STOCK`	409	Saldo insuficiente
`INVALID_STATE_TRANSITION`	409	Mudança de status não permitida
`EMPTY_CART`	422	Checkout com carrinho vazio
`PAYMENT_DECLINED`	402	Cobrança recusada
`RATE_LIMITED`	429	Limite do gateway
`UPSTREAM_UNAVAILABLE`	503	Serviço vizinho fora do ar
`INTERNAL`	500	Falha inesperada (detalhes só no log)

Erros do Prisma também são traduzidos: `P2002` vira `CONFLICT` e `P2025` vira `NOT_FOUND`.

Eventos tipados

`eventSchemas` associa cada nome de evento a um esquema. `bus.publish('order.placed', payload)` só aceita o payload certo, e o handler de `bus.subscribe` recebe o tipo exato. Na leitura, o envelope é validado de novo; payload inválido vai direto para a dead-letter.

CAPÍTULO 7

API REST

Envelope e convenções

- Base: `http://localhost:8080` (pela interface) ou `http://localhost:8000` (gateway direto).
- Valores em centavos (`priceCents`, `totalCents`) e datas em ISO 8601.
- Paginação com `page` e `pageSize` (máximo 100).
- Documentação interativa em `/api/docs` e especificação em `/api/openapi.json`, geradas dos contratos.

```
{ "ok": true, "data": { "items": [ ... ], "page": 1, "pageSize": 20, "total": 16, "totalPages": 1 } }
{ "ok": false, "error": { "code": "VALIDATION_FAILED", "message": "Dados inválidos",
  "details": [ { "path": "pageSize", "message": "Too big: expected number to be <=100" } ],
"requestId": "..." } }
```

Rotas

Método e rota	Acesso	O que faz
POST /oauth/authorize	público	Login com PKCE, devolve o código
POST /oauth/token	público	Troca código, refresh ou credenciais por token
POST /oauth/revoke	público	Revoga a família do refresh token
GET /oauth/jwks	público	Chaves públicas
POST /api/v1/auth/register	público	Cadastro de cliente
GET /api/v1/auth/me	usuário	Usuário atual
GET /api/v1/catalog/products	público	Busca: q, category, minPrice, maxPrice, sort, page, pageSize
GET /api/v1/catalog/products/{id}	público	Detalhe
POST /api/v1/catalog/products	admin	Cria produto
PATCH /api/v1/catalog/products/{id}	admin	Atualiza campos
DELETE /api/v1/catalog/products/{id}	admin	Desativa
GET/POST /api/v1/catalog/categories	público / admin	Lista e cria categorias
GET /api/v1/inventory?productIds=	público	Saldos
PUT /api/v1/inventory/{id}	admin	Define o disponível
GET /api/v1/cart	usuário	Carrinho com preços atuais
POST /api/v1/cart/items	usuário	Adiciona
PATCH /api/v1/cart/items/{productId}	usuário	Altera quantidade (0 remove)
DELETE /api/v1/cart	usuário	Esvazia

Método e rota	Acesso	O que faz
POST /api/v1/orders	usuário	Checkout (202)
GET /api/v1/orders	usuário	Lista; `scope=all` para admin
GET /api/v1/orders/{id}	usuário	Detalhe com histórico
POST /api/v1/orders/{id}/cancel	usuário	Cancela antes do pagamento
PATCH /api/v1/orders/{id}/status	admin	SHIPPED, DELIVERED ou CANCELLED
GET /api/v1/payments/orders/{orderId}	usuário	Pagamentos do pedido
GET /api/v1/notifications	usuário	Lista (`unread=true`)
GET /api/v1/notifications/unread	usuário	Contador
POST /api/v1/notifications/{id}/read	usuário	Marca como lida
POST /api/v1/notifications/read-all	usuário	Marca todas

Exemplos

```
curl "http://localhost:8080/api/v1/catalog/products?q=caf%C3%A9&sort=price_asc"

curl -X POST http://localhost:8080/oauth/token -H "Content-Type: application/json" \
  -d '{"grant_type":"refresh_token","refresh_token":"<refresh>","client_id":"cesto-web"}'

curl -X POST http://localhost:8080/api/v1/cart/items -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"productId":"<id>","quantity":2}'

curl -X PATCH http://localhost:8080/api/v1/orders/<id>/status -H "Authorization: Bearer $ADMIN" \
  -H "Content-Type: application/json" -d '{"status":"SHIPPED","note":"Coletado"}'
```

O arquivo `scripts/smoke.mjs` faz o fluxo inteiro (PKCE, carrinho, pedido, recusa, falta de estoque, refresh, administração) e serve de exemplo de cliente.

CAPÍTULO 8

Configuração

Variáveis do .env

Variável	Padrão	O que faz
APP_PORT	8080	Porta da interface
GATEWAY_PORT	8000	Porta da API direta
DB_USER / DB_PASSWORD	cesto / —	Credenciais do PostgreSQL
DB_HOST / DB_PORT	localhost / 5432	Banco no desenvolvimento sem Docker
REDIS_URL	redis://localhost:6379	Redis no desenvolvimento sem Docker
SERVICE_CLIENT_ID / SERVICE_CLIENT_SECRET	cesto-internal / —	Cliente OAuth2 entre serviços
AUTH_ISSUER / AUTH_AUDIENCE	cesto-auth / cesto-api	Emissor e público dos JWT
ACCESS_TOKEN_TTL	900	Validade do access token (s)
REFRESH_TOKEN_TTL	2592000	Validade do refresh token (s)
WEB_REDIRECT_URIS	http://localhost:8080/callback	Endereços de retorno permitidos (vírgulas)
CORS_ORIGINS	http://localhost:8080	Origens aceitas (vírgulas)
SEED_DEMO	true	Dados de demonstração
DEMO_PASSWORD	Cesto@2026	Senha das contas de demonstração
PAYMENT_DECLINE_ABOVE_CENTS	500000	Recusa cobranças acima deste valor
PAYMENT_DELAY_MS	800	Latência simulada do pagamento
RATE_LIMIT_MAX / RATE_LIMIT_WINDOW	300 / 1 minute	Limite por IP no gateway
LOG_LEVEL	info	fatal, error, warn, info, debug, trace ou silent

Cada serviço valida a própria configuração com Zod ao iniciar e para com uma mensagem clara se algo estiver faltando.

Checklist para produção

- Senhas fortes em `DB_PASSWORD` e `SERVICE_CLIENT_SECRET` (os scripts já geram).
- `SEED_DEMO=false` e conta de administrador própria.
- HTTPS na frente da interface e `WEB_REDIRECT_URIS` / `CORS_ORIGINS` com o domínio real.
- Gateway de pagamento real no lugar do simulado.
- Backup diário dos seis bancos e persistência do Redis (o compose já liga o AOF).

- Não publique a porta do gateway se só a interface for usada.

CAPÍTULO 9

Personalizar e estender

Adicionar uma rota

1. Declare o contrato em `packages/contracts/src/api.ts`, reaproveitando ou criando esquemas em `domains/`.
2. Registre no serviço: `route(app, api.catalog.novaRota, handler)`.
3. Rode `npm run build -w packages/contracts`. O handler, o cliente da interface e o OpenAPI passam a conhecer a rota.

Adicionar um evento

1. Acrescente o esquema em `eventSchemas` (`packages/contracts/src/events.ts`).
2. No serviço que publica, grave no outbox dentro da transação: `outbox(tx, createEnvelope('nome.evento', payload))`.
3. No serviço que consome, inclua o handler em `bus.subscribe({ 'nome.evento': ... })` usando a deduplicação por `processed_events`.

Adicionar um serviço

1. Copie um serviço parecido em `services/` e troque o nome no `package.json`.
2. Se tiver banco, crie `cesto_novo` em `docker/postgres/init.sql`, ajuste o `schema.prisma` e gere a migração com `npx prisma migrate dev` dentro da pasta do serviço.
3. Inclua o serviço na lista de `workspaces`, em `scripts/dev.mjs`, `scripts/generate.mjs` e `scripts/migrate.mjs`.
4. Adicione o bloco no `docker-compose.yml` e o prefixo em `services/gateway/src/routes.ts`.

Trocar o pagamento simulado

```
export class MeuProvedor implements PaymentGateway {
  readonly name = 'meu-provedor';
  async charge(request: ChargeRequest): Promise<ChargeResult> { ... }
  async refund(reference: string, amountCents: number): Promise<void> { ... }
}
```

Troque `new SimulatedGateway(...)` por `new MeuProvedor(...)` em `services/payments/src/main.ts`. A chave de idempotência enviada é o id do pedido.

Imagens, marca e interface

- Produtos aceitam `imageUrl`; sem imagem, a interface mostra um bloco com as iniciais.
- As cores da interface ficam em variáveis no topo de `apps/web/src/styles.css`.
- A logo fica em `docs/images/logo.svg` e `apps/web/public/favicon.svg`.

CAPÍTULO 10

Testes e qualidade

Testes automatizados

Comando	O que cobre
`npm test`	Contratos (tipos inferidos com `expectTypeOf`, URLs, DTOs, esquemas, eventos), núcleo HTTP (validação, envelope, JWT, papéis, escopos, mapeamento de erros), outbox, saga e preços, PKCE e escopos, carrinho, notificações
`TEST_REDIS_URL=...` `npm test`	Inclui o barramento com Redis real: entrega, confirmação e nova tentativa
`npm run typecheck`	TypeScript estrito em todos os pacotes e testes
`npm run smoke`	21 verificações ponta a ponta com a API rodando

Integração contínua

A cada push, o GitHub Actions instala, gera os clientes Prisma, compila, verifica os tipos, roda os testes com PostgreSQL e Redis reais, aplica as migrações, sobe os oito serviços, executa o teste ponta a ponta e constrói as duas imagens Docker.

CAPÍTULO 11

Solução de problemas

Problemas comuns

Sintoma	Causa provável	Como resolver
`defina DB_PASSWORD no .env`	`.env` ausente	Rode o script de início ou copie o `.env.example`
Porta 8080 ocupada	Outro programa na porta	Troque `APP_PORT` no `.env`
Login devolve `invalid_grant` sobre redirect_uri	Endereço diferente do cadastrado	Inclua o endereço em `WEB_REDIRECT_URIS` e reinicie o `auth`
Pedido parado em Reservando	`inventory` parado ou sem Redis	`docker compose logs inventory` e `/health`
401 em todas as rotas após trocar chaves	Tokens antigos	Saia e entre de novo
`Configuração inválida` ao iniciar	Variável ausente ou fora do formato	A mensagem diz qual; corrija no `.env`
Eventos na dead-letter	Handler falhou 5 vezes	`XRANGE cesto:events:dead-letter - +` mostra o evento e o motivo
Prisma não encontra o cliente	Clientes não gerados	`npm run generate`

Onde olhar

- `GET /health` no gateway mostra Redis e cada serviço.
- Os logs são JSON com `service` e `reqId`; filtre pelo `x-request-id` da resposta.
- No Redis: `XINFO GROUPS cesto:events:order.placed` mostra pendências por serviço.
- No banco: `SELECT * FROM outbox_events WHERE published_at IS NULL` mostra eventos que ainda não saíram.